

NEURAL NETWORKS

Presented By
Rajatha

Table of Contents

Neural Networks:

- What is a neural network?
- Models of a Neuron
- Activation functions
- Network Architectures
- Knowledge representation
- Learning Process

Deep Feedforward Networks:

- Multilayer Perceptron
- Example: Learning XOR
- Gradient-Based Learning
- Hidden Units
- Architecture Design
- Back-Propagation Algorithm.

Introduction: Why Deep Learning?

The Limits of Traditional Programs

Traditional computer programs are excellent at tasks that can be *clearly defined with rules* – for example:

- Adding two numbers,
- Sorting a list alphabetically,
- Computing financial interest or tax values.

These are rule-based problems – the programmer writes the exact sequence of steps to follow.

But what happens when we ask the computer to do something like *recognize handwritten digits*?

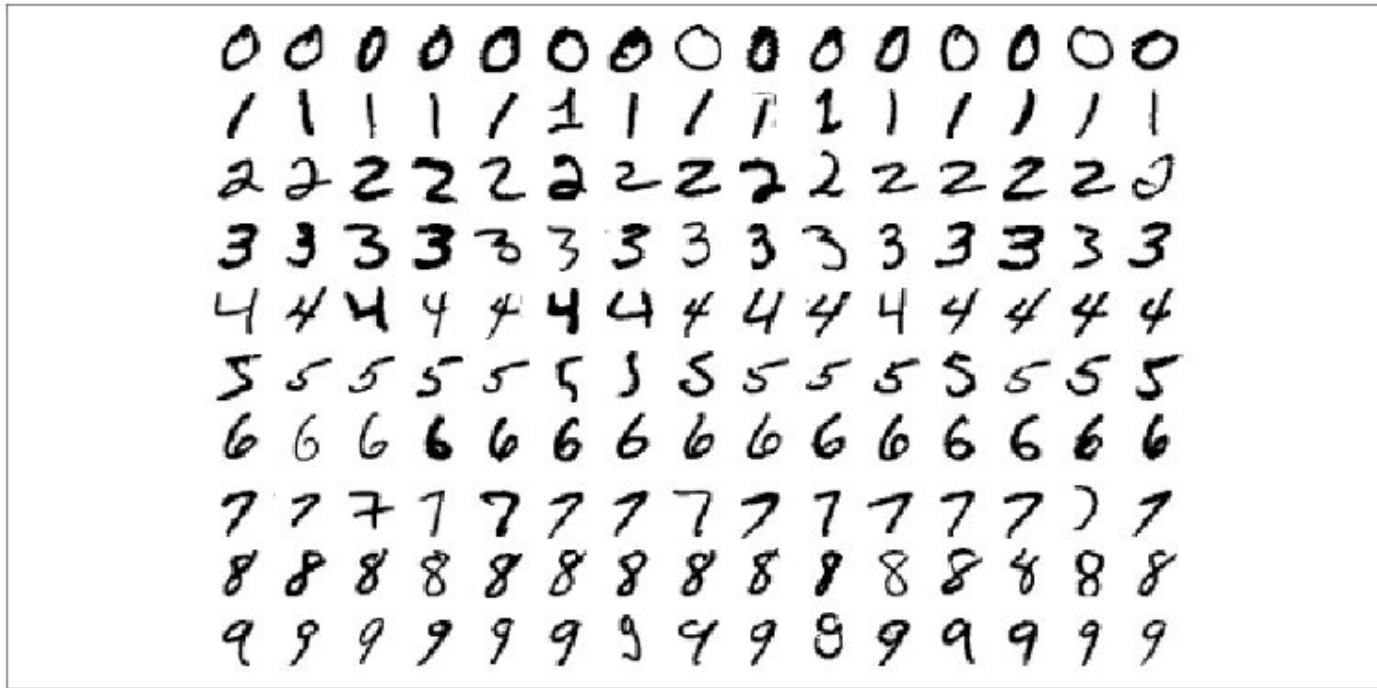


Fig: Image from MNIST handwritten digit dataset

- At first glance, this looks easy to humans; we instantly know the top row is all zeros, the next all ones, and so on.
- But if you try to write a traditional program to identify these digits, you'll quickly run into trouble.

Why Rules Fail for Handwriting?

Imagine trying to write an algorithm to detect the digit “0.”

You might start with a simple rule:

“If the image has a single closed loop, classify it as zero.”

Seems fine — until you realize:

- Some zeros aren’t fully closed (people write sloppy zeros),
- Some sixes also have a closed loop,
- Some zeros are tilted or thick,
- Some have gaps or dots in the middle.

Your single rule will **fail** for hundreds of variations.

You could try adding more rules:

- If it's round → zero
- If it has a small tail → six
- If it has two loops → eight
- If there's a vertical line → one

But soon, you'll need *hundreds or even thousands of conditions*, and still, you'll miss edge cases.

The program becomes a brittle ,that works only for specific examples ,not the general concept.

What Humans Do Differently?

Humans don't rely on explicit rules when recognizing digits. Instead, we rely on **pattern recognition** learned from experience:

- We've seen thousands of examples of each digit in different styles.
- Our brain automatically extracts visual features like curves, loops, line thickness, and spatial relationships.
- Even if a digit is messy, we can still guess it correctly based on learned patterns.

So, the key question becomes:

Can a computer *learn patterns from data* instead of being programmed with rules?

That's the starting point of machine learning and eventually deep learning!!!

From Rules to Learning

Instead of hand-coding every rule, we now **show the computer examples:**

- Thousands of images of “0,” “1,” “2,” ... “9”
- Along with their correct labels

Then, the computer learns its own internal rules, adjusting mathematical parameters (weights) to minimize the difference between its predictions and the true labels.

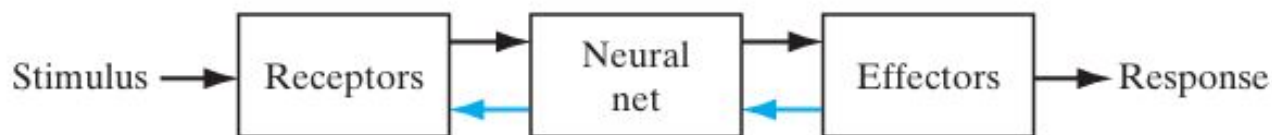
This approach is called **supervised learning**, and when we use multi-layered neural networks for this, it becomes **deep learning**.

THE HUMAN BRAIN

- The brain is the most incredible organ in the human body.
- It perceives every sight, sound, smell, taste, touch, stores memories, emotions and dream.
- The brain is a highly complex, nonlinear, and parallel computer (information-processing system).
- It has the capability to organize its structural constituents, known as neurons, so as to perform certain computations (e.g., pattern recognition, perception, and motor control) many times faster than the fastest digital computer in existence today.

- Consider, for example, human vision, which is an information-processing task.
- It is the function of the visual system to provide a representation of the environment around us and, more important, to supply the information we need to interact with the environment.
- To be specific, the brain routinely accomplishes perceptual recognition tasks (e.g., recognizing a familiar face embedded in an unfamiliar scene) in approximately 100-200 ms, whereas tasks of much lesser complexity take a great deal longer on a powerful computer.
- The human nervous system may be viewed as a three-stage system.
- Central to the system is the brain, represented by the neural (nerve) net, which continually receives information, perceives it, and makes appropriate decisions.

- Two sets of arrows are shown in the figure. Those pointing from left to right indicate the forward transmission of information-bearing signals through the system.
- The arrows pointing from right to left (shown in red) signify the presence of feedback in the system.
- The receptors convert stimuli from the human body or the external environment into electrical impulses that convey information to the neural net (brain).
- The effectors convert electrical impulses generated by the neural net into discernible responses as system outputs.



Block diagram representation of nervous system.

What is a Neuron?

- The foundational unit of the human brain is the neuron.
- A tiny piece of the brain, about the size of grain of rice, contains over 10,000 neurons, each of which forms an average of 6,000 connections with other neurons.
- It's this massive biological network that enables us to experience the world around us.
- At its core, the neuron is optimized to receive information from other neurons, process this information in a unique way, and send its result to other cells.
- The neuron receives its inputs along antennae like structures called dendrites.
- Each of these incoming connections is dynamically strengthened or weakened based on how often it is used .
- The strength of each connection that determines the contribution of the input to the neuron's output.

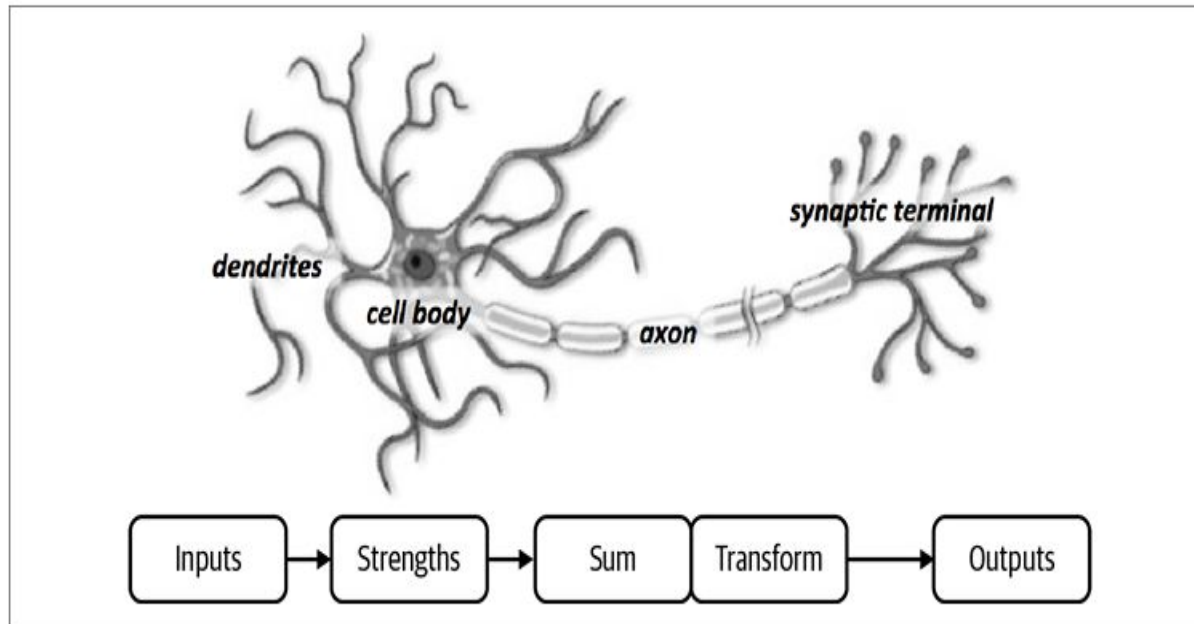
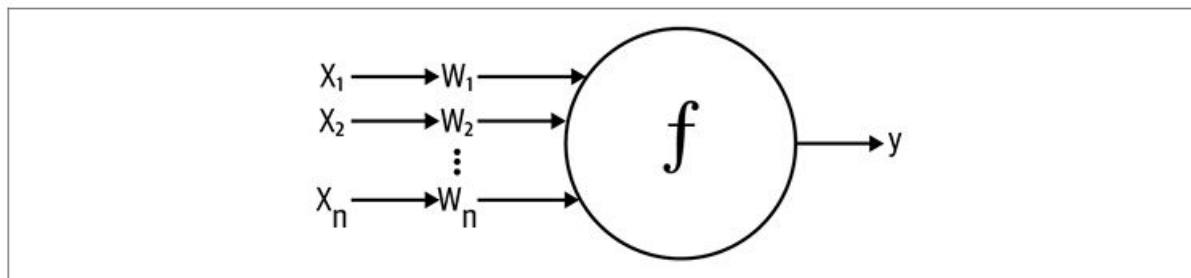


Fig: Functional Description of Biological Neuron Network

After being weighted by the strength of their respective connections, the inputs are summed together in the *cell body*. This sum is then transformed into a new signal that's propagated along the cell's *axon* and sent off to other neurons

- We can translate this functional understanding of the neurons in our brain into an artificial model that we can represent on our computer.
- Just as in biological neurons, our artificial neuron takes in some number of inputs, $x_1, x_2, \dots, x_n$, each of which is multiplied by a specific weight, $w_1, w_2, \dots, w_n$.
- These weighted inputs are, as before, summed to produce the logit of the neuron, $z = \sum_{i=1}^n w_i x_i$.
- In many cases, the logit also includes a bias, which is a constant.
- The logit is then passed through a function f to produce the output : $y = f(z)$.
- This output can be transmitted to other neurons.



$$\mathbf{x} = [x_1, x_2, \dots, x_n]$$

$$\mathbf{w} = [w_1 \ w_2 \ \dots \ w_n].$$

$$y = f(\mathbf{x} \cdot \mathbf{w} + b)$$

NEURAL NETWORK

- A neural network is a machine designed to model the way the brain performs a particular task or function of interest.
- The network is usually implemented using electronic components or simulated in software on a digital computer.

A neural network is a massively parallel distributed processor made up of simple processing units that has a natural propensity for storing experiential knowledge and making it available for use.

It resembles the brain in two respects:

1. Knowledge is acquired by the network from its environment through a learning process.
2. Interneuron connection strengths, known as synaptic weights, are used to store the acquired knowledge.
 - The procedure used to perform the learning process is called a *learning algorithm*.
 - The Learning Algorithm is used to modify the synaptic weights of the network in an orderly fashion to attain a desired design objective.

Benefits of Neural Networks

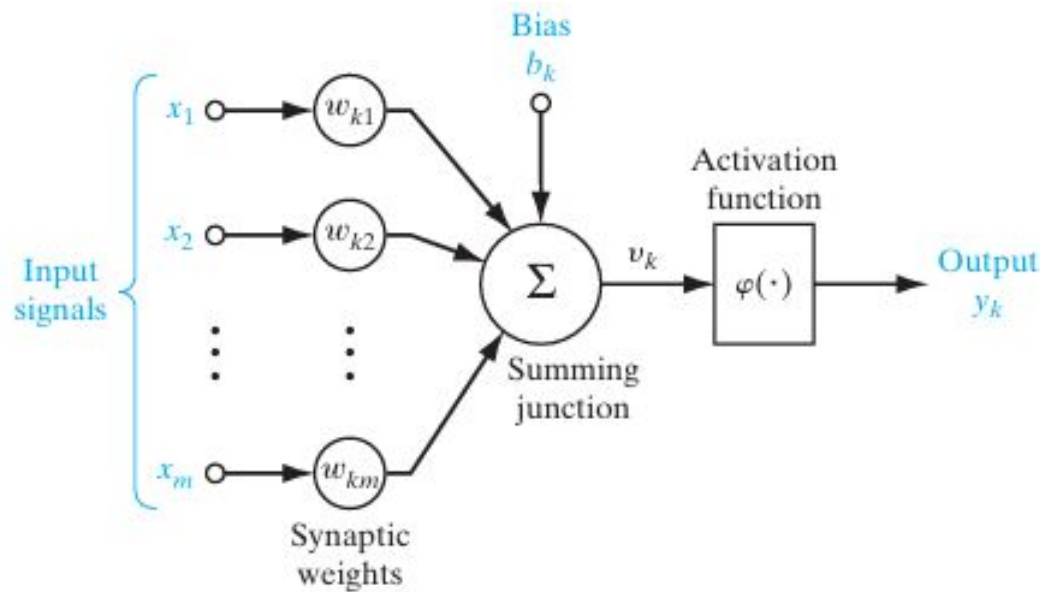
A neural network derives its computing power through :

1. A massively parallel distributed structure
2. It's ability to learn and therefore generalize
 - *Generalization* refers to the neural network's production of reasonable outputs for inputs not encountered during training (learning).
 - These two information processing capabilities make it possible for neural networks to find good approximate solutions to complex (large-scale) problems that are *intractable*.
 - Neural Networks, cannot provide solutions by working individually. Rather, they need to be integrated into a consistent system engineering approach.

- A complex problem of interest is decomposed into a number of relatively simple tasks, and neural networks are assigned a subset of the tasks that match their inherent capabilities.
- Neural networks offer the following useful properties and capabilities:
 1. Nonlinearity
 2. Input–Output Mapping.
 3. Adaptivity
 4. Evidential Response
 5. Contextual Information
 6. Fault Tolerance
 7. VLSI Implementability
 8. Uniformity of Analysis and Design
 9. Neurobiological Analogy

Models of a Neuron

A neuron is an information-processing unit that is fundamental to the operation of a neural network.



Nonlinear model of a neuron, labeled k .

We identify three basic elements of the neural model:

1. A set of synapses, or connecting links, each of which is characterized by a weight or strength of its own.
2. Specifically, a signal x_j at the input of synapse j connected to neuron k is multiplied by the synaptic weight w_{kj} . (Unlike the weight of a synapse in the brain, the synaptic weight of an artificial neuron may lie in a range that includes negative as well as positive values.)
3. An adder for summing the input signals, weighted by the respective synaptic strengths of the neuron; the operations described here constitute a linear combiner.
4. An activation function for limiting the amplitude of the output of a neuron.
5. The activation function is also referred to as a squashing function, in that it squashes (limits) the permissible amplitude range of the output signal to some finite value
6. Neural Networks also include an externally applied bias, denoted by b_k . The bias b_k has the effect of increasing or lowering the net input of the activation function, depending on whether it is positive or negative.

In mathematical terms, we may describe the neuron k by writing the pair of equations:

$$u_k = \sum_{j=1}^m w_{kj} x_j \quad (1)$$

and

$$y_k = \varphi(u_k + b_k) \quad (2)$$

where $x_1, x_2, \dots, x_m$ are the input signals; $w_{k1}, w_{k2}, \dots, w_{km}$ are the respective synaptic weights of neuron k ; u_k (not shown in Fig. 5) is the *linear combiner output* due to the input signals; b_k is the bias; $\varphi(\cdot)$ is the *activation function*; and y_k is the output signal of the neuron. The use of bias b_k has the effect of applying an *affine transformation* to the output u_k of the linear combiner in the model of Fig. 5, as shown by

$$v_k = u_k + b_k \quad (3)$$

In particular, depending on whether the bias b_k is positive or negative, the relationship between the induced local field, or activation potential, v_k of neuron k and the linear combiner output u_k .

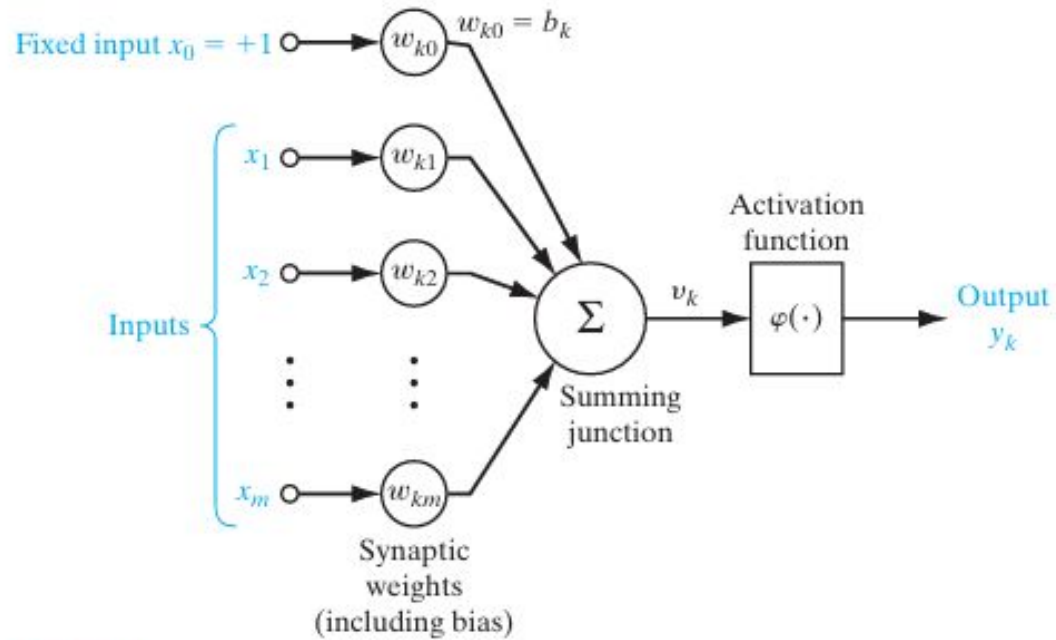


FIGURE 7 Another nonlinear model of a neuron; w_{k0} accounts for the bias b_k .

In a **deterministic neuron** (like the McCulloch-Pitts model), the output is *completely fixed* for a given input.

- If the input signal is strong enough → neuron fires (output = 1)
- If it's weak → neuron stays off (output = -1)
- There's no randomness — the neuron behaves the same way every time for the same input.
- In real life (especially in biological brains or noisy environments), neurons don't always behave predictably.
Even with the same input, sometimes they fire, sometimes they don't — due to **synaptic noise** or **uncertainty**.
- To mimic that natural randomness, we make the neuron **stochastic** — meaning its firing becomes **probabilistic**.

We say:

$$x = \begin{cases} +1 & \text{with probability } P(v) \\ -1 & \text{with probability } 1 - P(v) \end{cases}$$

Here:

- x = neuron's state (on = +1, off = -1)
- v = total input signal to the neuron (called *local field*)
- $P(v)$ = probability that the neuron fires

◆ The Sigmoid Function for Probability

We define $P(v)$ as a smooth curve that increases as v increases:

$$P(v) = \frac{1}{1 + e^{-v/T}}$$

- When v is large $\rightarrow P(v) \approx 1$ (neuron almost certainly fires)
- When v is small or negative $\rightarrow P(v) \approx 0$ (neuron likely stays off)

◆ Role of T — the “Pseudo Temperature”

- T controls **how random** the neuron is.
- If T is high $\rightarrow$ neuron is *noisy* (more random). Even weak inputs might make it fire.
- If T is low $\rightarrow$ neuron is *more deterministic*.
- As $T \rightarrow 0 \rightarrow$ randomness disappears, and the neuron behaves like the **normal deterministic McCulloch–Pitts neuron**.

Activation Function

What Is an Activation Function?

An **activation function** decides what the output of a neuron should be after combining all its weighted inputs. It basically answers:

“Should this neuron fire or stay quiet, and how strongly should it fire?”

It converts the raw input v (also called the **induced local field**) into an **output** y that is passed to the next layer.

$$y=f(v)$$

Types of Activation Functions

There are two **basic types**— **Threshold** and **Sigmoid**, and then several **modern ones** used today (ReLU, Leaky ReLU, etc.).

Threshold (Step or Heaviside) Function

Formula:

$$f(v) = \begin{cases} 1, & v \geq 0 \\ 0, & v < 0 \end{cases}$$

Meaning:

If the input (v) crosses a certain limit (threshold), neuron fires (1);
otherwise, it stays off (0).

In engineering, this form of a threshold function is commonly referred to as a Heaviside function.

Correspondingly, the output of neuron k employing such a threshold function is expressed as:

$$y_k = \begin{cases} 1 & \text{if } v_k \geq 0 \\ 0 & \text{if } v_k < 0 \end{cases}$$

where v_k is the induced local field of the neuron; that is,

$$v_k = \sum_{j=1}^m w_{kj} x_j + b_k$$

Characteristics:

- Output: 0 or 1
- **Not continuous**, not smooth
- Looks like a sharp step — jumps suddenly from 0 to 1.
- Used in **McCulloch-Pitts model** (the oldest neuron model)
- Represents **all-or-none behavior** (either fires or not)

Sigmoid (Logistic) Function

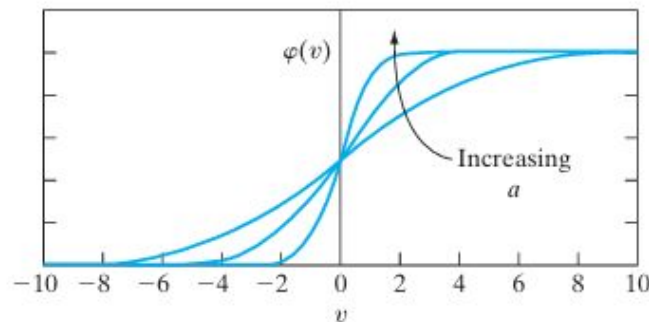
The sigmoid function is defined as an activation function that maps any real value to the interval $[0,1]$ and is expressed mathematically as

Formula:

$$f(v) = \frac{1}{1 + e^{-av}}$$

Meaning:

- Converts input into a smooth S-shaped curve (values between 0 and 1).
- a = slope parameter $\rightarrow$ controls how steep the curve is.



Characteristics:

- Output range: 0 to 1
- **Continuous and differentiable** (good for backpropagation)
- As $a \rightarrow \infty$, it becomes a threshold function (hard step).
- Commonly used in early neural networks.

Use:

- In logistic regression and simple feedforward neural nets for **probability-like outputs**.

A threshold function assumes the value of 0 or 1, a sigmoid function assumes a continuous range of values from 0 to 1.

sigmoid function is differentiable, whereas the threshold function is not.

Imagine a medical diagnosis system:

- If probability of disease = 0.9 $\rightarrow$ “Highly likely”
- If 0.5 $\rightarrow$ “Uncertain”
- If 0.1 $\rightarrow$ “Unlikely”

Signum Function

It is sometimes desirable to have the activation function range from -1 to 1, in which case, the activation function is an odd function of the induced local field.

Formula:

$$f(v) = \begin{cases} +1, & v > 0 \\ 0, & v = 0 \\ -1, & v < 0 \end{cases}$$

Meaning:

Similar to threshold, but allows **negative output (-1)** when input is negative.

Use:

- Used when symmetric output (positive and negative) is needed.

Hyperbolic Tangent (tanh) Function

Formula:

$$f(v) = \tanh(v) = \frac{e^v - e^{-v}}{e^v + e^{-v}}$$

Range:

-1 to +1

Characteristics:

- Smooth and differentiable
- Output centered around 0 (unlike sigmoid which is always positive)
- Helps the model learn faster because it balances positive and negative signals.

Use:

- Widely used in hidden layers of neural networks before ReLU became popular.

Question:

For each situation below, choose the **best activation function** and justify why:

1. Detecting whether an image contains a cat (yes/no).
2. Predicting probability of a patient having diabetes.
3. Controlling brightness in an image enhancement algorithm.
4. Deep CNN for object detection.

The sigmoid slope at the origin ($v = 0$) is given by $a/4$.

For slope parameters $a = 1, 2, 4$:

- a) Calculate the slope value.
- b) Explain how increasing a affects the steepness of the curve.

Formula:

$$f(v) = \max(0, v)$$

Meaning:

- If input $> 0 \rightarrow$ output = input
- If input $\leq 0 \rightarrow$ output = 0

Characteristics:

- Fast and simple
- Prevents gradient saturation (faster learning)
- Not differentiable at $v = 0$, but works well in practice
- Most common activation in CNNs and deep learning models

Use:

- Used in almost all **modern deep networks**

Neural Network Architectures

The way neurons are structured directly affects how the network learns (its learning algorithm).

Single-Layered Neural Networks: In a layered neural network, the neurons are organized in the form of layers. In the simplest form of a layered network,

There are **two layers of nodes**:

Input layer: just passes input signals into the network (no computation).

Output layer: contains the neurons that perform the actual computation.

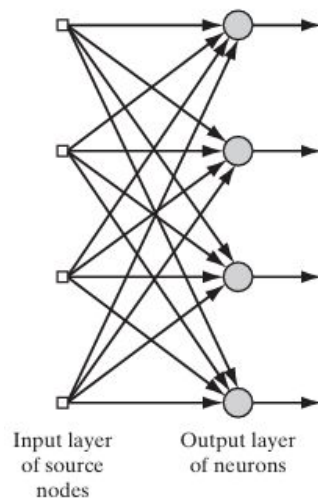


FIGURE 15 Feedforward network with a single layer of neurons.

Imagine you input four features (like temperature, pressure, humidity, and wind speed), and the network directly gives one output (like “rain yes/no”).
Each input connects directly to the output neurons.

Multilayer Feedforward Network

The second class of a feedforward neural network distinguishes itself by the presence of one or more hidden layers, whose computation nodes are correspondingly called hidden neurons or hidden units.

The term “hidden” refers to the fact that this part of the neural network is not seen directly from either the input or output of the network.

- **Has three or more layers:**

- **Input layer** → source of signals
- **Hidden layer(s)** → do intermediate processing
- **Output layer** → gives final result

So, information flows like:

Input → *Hidden layer(s)* → Output

Each hidden layer extracts **higher-level features** from the data.

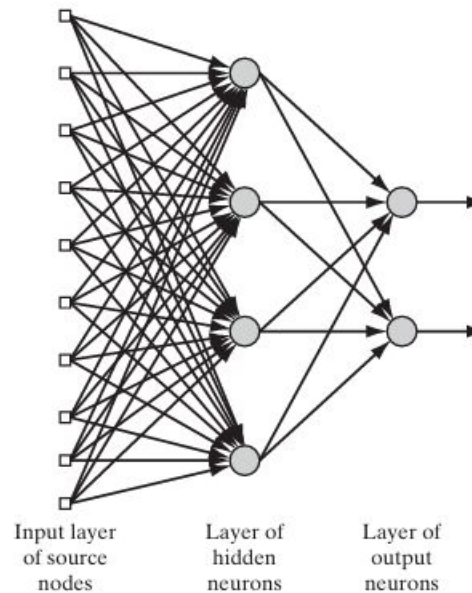
Example:

In an image network, early layers detect **edges**,
later layers detect **shapes or objects**.

A **10-4-2 network** means:

- 10 input nodes
- 4 hidden neurons
- 2 output neurons

FIGURE 16 Fully connected feedforward network with one hidden layer and one output layer.



Recurrent Networks

A recurrent neural network distinguishes itself from a feedforward neural network in that it has at least one feedback loop.

For example, a recurrent network may consist of a single layer of neurons with each neuron feeding its output signal back to the inputs of all the other neurons,

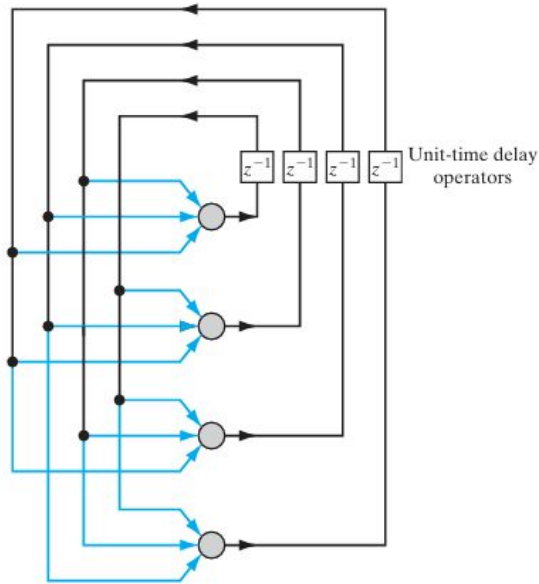


FIGURE 17 Recurrent network with no self-feedback loops and no hidden neurons.

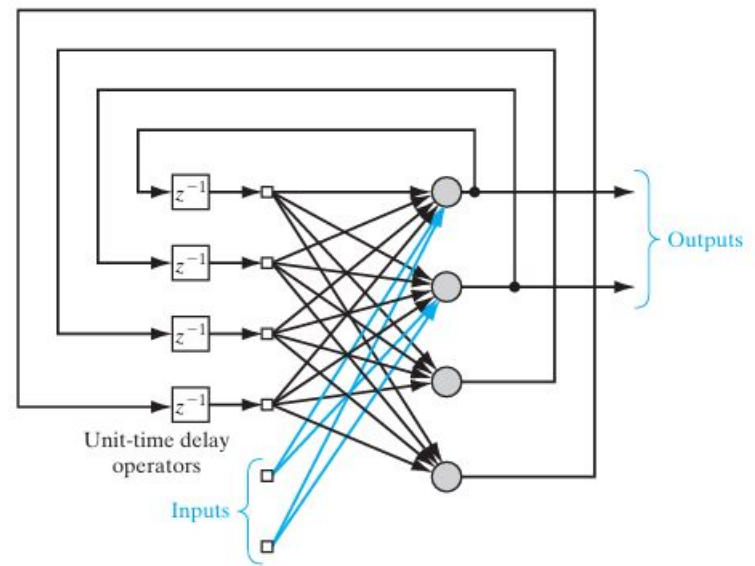


FIGURE 18 Recurrent network with hidden neurons.

Structure:

- Different from feedforward networks because it has **feedback loops**.
- That means **outputs are fed back** into earlier layers (or even the same layer).

What this does:

Makes the network capable of remembering **past information** — it has **memory**.
Useful for **time-dependent data** (like speech, weather, or stock prices).

Examples:

- **Simple recurrent network:**

Each neuron sends its output to others in the same layer (but not to itself).

- **Recurrent network with hidden neurons:**

Hidden and output neurons both send feedback signals.

Knowledge Representation

The Learning Process

How do neural networks work?

Each individual node as its own **linear regression** model, composed of input data, weights, a bias (or threshold), and an output.

The formula would look something like this:

$$\sum w_i x_i + \text{bias} = w_1 x_1 + w_2 x_2 + w_3 x_3 + \text{bias}$$

$$\text{Output} = f(x) = 1 \quad \text{if } \sum w_i x_i + b \geq 0;$$

$$\text{Output} = f(x) = 0 \quad \text{if } \sum w_i x_i + b < 0$$

- 
- Once an input layer is determined, weights are assigned.
 - These weights help determine the importance of any given variable, with larger ones contributing more significantly to the output compared to other inputs.
 - All inputs are then multiplied by their respective weights and then summed.
 - Afterward, the output is passed through an activation function, which determines the output.
 - If that output exceeds a given threshold, it “fires” (or activates) the node, passing data to the next layer in the network.
 - This results in the output of one node becoming in the input of the next node.
 - This process of passing data from one layer to the next layer defines



Example

whether you should go surfing (Yes: 1, No: 0).

The decision to go or not to go is our predicted outcome, or $\hat{y}$.
Let's assume that there are three factors influencing your decision-making:

1. Are the waves good? (Yes: 1, No: 0)
2. Is the line-up empty? (Yes: 1, No: 0)
3. Has there been a recent shark attack? (Yes: 0, No: 1)



Then, let's assume the following, giving us the following inputs:

$X_1 = 1$, since the waves are pumping

$X_2 = 0$, since the crowds are out

$X_3 = 1$, since there hasn't been a recent shark attack

Now, we need to assign some weights to determine importance.

Larger weights signify that particular variables are of greater importance to the decision or outcome.



Finally,

- we'll also assume a threshold value of 3, which would translate to a bias value of -3 .
- With all the various inputs, we can start to plug in values into the formula to get the desired output.
- $\hat{Y} = (1*5) + (0*2) + (1*4) - 3 = 6$

- 
- If we use the activation function, we can determine that the output of this node would be 1, since 6 is greater than 0.
 - In this instance, you would go surfing;

But if we adjust the weights or the threshold, we can achieve different outcomes from the model.

When we observe one decision, like in the above example, we can see how a neural network could make increasingly complex decisions

Models of a Neuron

- A neuron is an information-processing unit that is fundamental to the operation of a neural network.
- The block diagram of neural network is

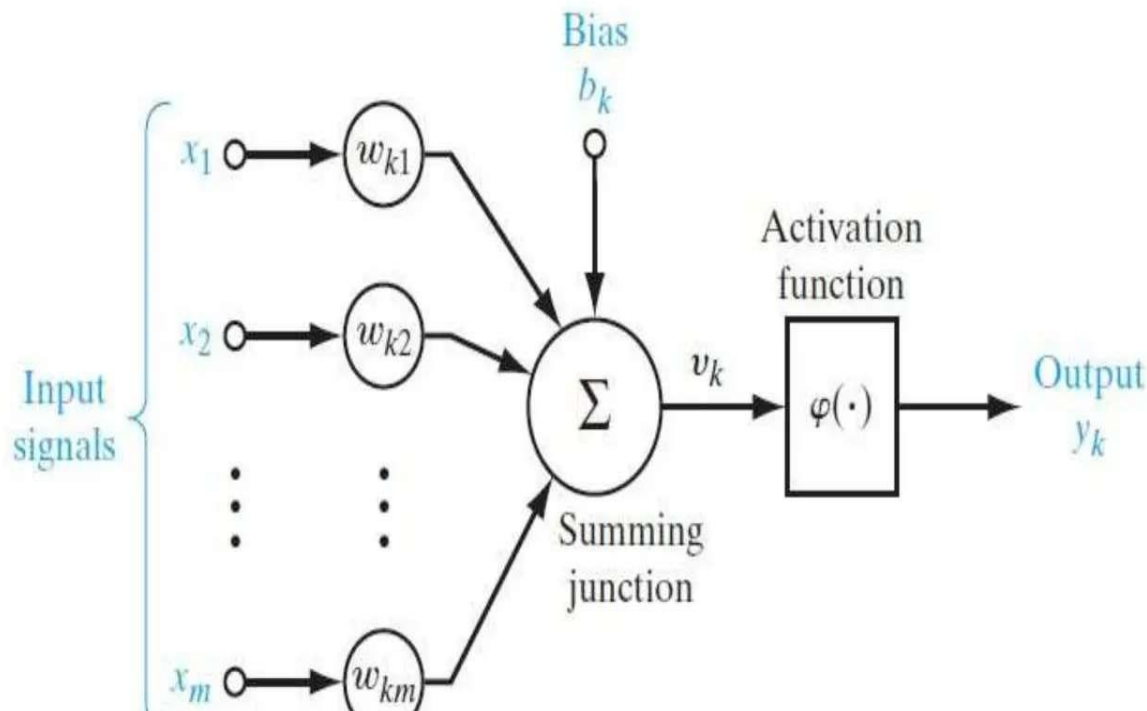


FIGURE 5 Nonlinear model of a neuron, labeled k .



We identify three basic elements of the neural model:

- 1. A set of synapses, or connecting links, each of which is characterized by a weight or strength of its own.
- Specifically, a signal x_j at the input of synapse j connected to neuron k is multiplied by the synaptic weight w_{kj} .
- 2. An **adder** for summing the input signals, weighted by the respective synaptic strengths of the neuron; the operations described here constitute a linear combiner.
- 3. An **activation function** for limiting the amplitude of the output of a neuron.
- The activation function is also referred to as a squashing function, in that it squashes (limits) the permissible amplitude range of the output signal to some finite range.

- The normalized amplitude range of the output of a neuron is written as the closed unit interval $[0,1]$, or, alternatively, $[-1,1]$.
- The above neural model also includes an externally applied bias, denoted by **bk**.
- The **bias bk** has the effect of increasing or lowering the net input of the activation function, depending on whether it is positive or negative, respectively
- In mathematical terms, we may describe the neuron k depicted by writing the pair of equations

$$u_k = \sum_{j=1}^m w_{kj} x_j \quad (1)$$

and

- 
- where
 - $x_1, x_2, \dots, x_m$ are the input signals;
 - $w_{k1}, w_{k2}, \dots, w_{km}$ are the respective synaptic weights of neuron k ;
 - u_k is the linear combiner output due to the input signals;
 - b_k is the bias;
 - $\varphi(\cdot)$ is the activation function; and
 - y_k is the output signal of the neuron.

- Types of Activation Function

- The activation function, denoted by $\varphi(v)$, defines the output of a neuron in terms of the induced local field v
- We identify two basic types of activation functions:

1.Threshold Function. For this type of activation function, described in Fig

